Sparse Merkle Trees with Poseidon Hash for Enterprise ZK Validium State Management

Base Computing S.A.S. Research Team
Medellin, Colombia
social@basecomputing.com.co

***Abstract*—Enterprise blockchain systems face a fundamental tension between verifiability and data privacy. Zero-knowledge validium architectures resolve this by maintaining state off-chain and submitting validity proofs to a Layer 1 chain, but their performance depends critically on the state commitment structure. This paper presents the design, implementation, and empirical evaluation of a depth-32 Sparse Merkle Tree (SMT) using the Poseidon hash function over the BN128 scalar field, purpose-built for the Basis Network enterprise ZK validium node on Avalanche. We evaluate the implementation across tree sizes from 100 to 100,000 entries, measuring insert latency, proof generation time, proof verification time, and memory consumption. At 100,000 entries, the SMT achieves mean insert latency of 1.825 ms (5.5× below the 10 ms target), proof generation in 0.018 ms (278× below target), and proof verification in 1.744 ms (within the 2 ms target). Insert latency is constant across tree sizes, confirming O(d) complexity independent of entry count. A comparative benchmark demonstrates that Poseidon provides a 4.97× speedup over MiMC in native JavaScript execution, consistent with its lower R1CS constraint count (240 vs. 340 per hash). The implementation maintains full compatibility with the BN128 scalar field via circomlibjs, enabling direct integration with Circom/SnarkJS Groth16 proof circuits. These results validate the SMT as a viable foundation for enterprise ZK validium state management, with identified optimization paths for scaling beyond 100,000 entries.**

***Index Terms*—Sparse Merkle Tree, Poseidon hash, zero-knowledge proofs, validium, enterprise blockchain, BN128**

I. Introduction

Enterprise blockchain adoption remains constrained by a fundamental tension between transparency and privacy. Public blockchains provide verifiability and auditability but expose all transaction data, making them unsuitable for enterprises that must protect trade secrets, supplier relationships, and competitive pricing. Private blockchains preserve confidentiality but sacrifice the trust guarantees that make blockchain compelling in the first place. Zero-knowledge (ZK) proof systems offer a resolution to this tension: an enterprise can prove the correctness of its state transitions without revealing the underlying data [1].

A critical architectural decision in any ZK-based enterprise system is the choice of state commitment structure. The state tree must satisfy three simultaneous requirements: (1) it must support efficient membership and non-membership proofs for on-chain verification, (2) it must operate natively over the finite field used by the ZK proof system to minimize circuit complexity, and (3) it must scale to enterprise workloads—hundreds of thousands of state entries—with sub-second operation latency. These requirements are not independently satisfiable; the choice of hash function, tree structure, and field arithmetic are deeply coupled.

This paper presents the design, implementation, and empirical evaluation of a depth-32 Sparse Merkle Tree (SMT) using the Poseidon hash function [2] over the BN128 scalar field, purpose-built for the Basis Network enterprise ZK validium node. The Basis Network is an Avalanche Layer 1 blockchain serving Latin American enterprises, where each enterprise operates a validium node that maintains private state off-chain and submits ZK validity proofs to the L1. The SMT serves as the foundational state structure upon which all subsequent components—transaction queuing, batch aggregation, proof generation, and L1 submission—depend.

Our evaluation demonstrates that the proposed SMT implementation meets all target performance requirements with significant margin: insert latency of 1.825 ms (5.5× below the 10 ms target), proof generation in 0.018 ms (278× below the 5 ms target), and proof verification in 1.744 ms (within the 2 ms target), all measured at 100,000 entries on a standard development machine. We further establish that Poseidon provides a 4.97× speedup over MiMC in native JavaScript execution, consistent with the theoretical prediction based on relative constraint counts.

II. Related Work

A. ZK-Friendly Hash Functions

The design of hash functions optimized for zero-knowledge proof systems has been an active area of research since the introduction of MiMC [3], which minimized multiplicative complexity through high-degree S-boxes over prime fields. Poseidon [2] advanced this line of work with the HADES permutation strategy, combining full and partial S-box rounds to achieve 240 R1CS constraints per 2-to-1 hash over BN254—a 29% reduction from MiMC's 340 constraints. Poseidon2 [4] further optimizes native computation speed while maintaining the same constraint count.

Guohanze et al. [5] provide the most comprehensive cross-system benchmark to date, measuring constraint counts and proving times across hash functions and SNARK backends on EVM-compatible chains. Their results confirm Poseidon's 240-constraint figure for R1CS/Groth16 on BN254 and report Groth16 proof sizes of approximately 800 bytes with verification gas of ∼219,000 on Ethereum. Nielsen [6] extends this

with empirical measurements showing Poseidon proof sizes of 112 KB versus MiMC's 203 KB in the SnarkJS pipeline.

Recent cryptanalysis by Grassi et al. [7] identified inaccuracies in original round-count security estimates through Gröbner basis attacks exploiting subspace trails. However, the Ethereum Foundation's Poseidon Cryptanalysis Initiative [8] confirms the overall security of Poseidon with recommended parameters, and no practical attacks have been demonstrated against production configurations.

The Rescue hash function [9] offers higher security margins at approximately 600 R1CS constraints per hash, representing a 2.5× cost increase over Poseidon with no compensating advantage for our use case. Traditional hash functions (SHA-256 at $\sim$25,000 constraints, Keccak-256 at $\sim$150,000 constraints) remain infeasible for in-circuit use at the scale required for Merkle tree verification.

B. Sparse Merkle Trees

Sparse Merkle Trees extend the original Merkle tree construction [10] to support deterministic key-value storage with both membership and non-membership proofs. Dahlberg et al. [11] formalized efficient non-membership proofs and caching strategies. Haider [12] introduced compact representations that reduce proof sizes. Baylina and Bellés [13] standardized the SMT construction for ZK applications at the ZKProof Standards Workshop, establishing conventions adopted by Iden3 and subsequently by Polygon.

Chen et al. [14] address batch update optimization for rollup contexts, demonstrating shared-path computation techniques that reduce amortized hash operations per insert. Reilabs [15] push the scaling frontier to billion-key trees in Rust, reporting per-insert latencies of 63–193 μs across 4–16 cores with multi-threaded parallelism.

C. Production Deployments

Table I summarizes the state tree implementations in major production ZK systems.

TABLE I
STATE TREE IMPLEMENTATIONS IN PRODUCTION ZK SYSTEMS.

System	Tree Type	Hash	Depth
Polygon zkEVM [16]	Sparse Merkle Trie	Poseidon	256
Scroll [17]	Poseidon SMT	Poseidon	256
Iden3 [18]	Sparse Merkle Tree	Poseidon	256
Semaphore v4 [19]	Lean IMT	Poseidon	Dynamic

Notably, all major production ZK systems have converged on Poseidon as the hash function of choice. Semaphore's migration from MiMC (v2) to Poseidon (v3) halved proving times [19], providing direct empirical evidence of the performance advantage in circuit proving. Polygon zkEVM and Scroll both use depth-256 trees to support the full Ethereum address space; our depth-32 design targets a smaller enterprise address space ($2^{32} \approx 4.3$ billion entries) in exchange for proportionally lower constraint costs in circuit verification.

III. System Model and Design

A. Enterprise Validium Architecture

The Basis Network operates as a permissioned Avalanche Layer 1 (Subnet-EVM) with zero transaction fees. Each registered enterprise runs an off-chain validium node that processes transactions privately, maintains a Sparse Merkle Tree of enterprise state, and periodically submits ZK validity proofs to the L1. The L1 verifies these proofs and stores the resulting state roots, providing a public commitment to each enterprise's state without revealing its contents.

The trust model assumes an honest L1 (Avalanche consensus with sub-second finality) and potentially adversarial enterprises. The SMT must therefore provide:

- **Soundness**: A valid Merkle proof can only be produced for data actually in the tree.
- **Completeness**: Every entry in the tree has a valid proof.
- **Determinism**: The same set of key-value pairs always produces the same root hash.
- **Non-membership**: The tree must prove that a key does *not* exist, which is critical for double-spend prevention.

B. Sparse Merkle Tree Design

We implement a binary SMT of fixed depth $d = 32$, providing $2^{32} \approx 4.3 \times 10^9$ addressable leaf positions. The tree stores key-value pairs where keys are field elements in $\mathbb{F}_p$ (the BN128 scalar field, $p =$ 21888242871839275222246405745257275088548364400416034343698204

a) Node storage.: Only non-empty nodes are stored in a hash map keyed by $(level, index)$. Empty subtree hashes are precomputed at initialization: $h_0 = 0$ (empty leaf), $h_i = \text{Poseidon}(h_{i-1}, h_{i-1})$ for $i \in [1, d]$. This precomputation avoids redundant hashing of the $2^{32} - n$ empty leaves, where n is the number of populated entries.

b) Key mapping.: Keys are mapped to leaf indices by extracting the lower d bits of the key's field representation. For keys derived from Poseidon hashes of application identifiers (e.g., Poseidon($\mathit{enterpriseId}, \mathit{recordId}$)), this provides uniform distribution across the tree.

c) Leaf hashing.: Non-empty leaves store Poseidon$(key, value)$; empty leaves store 0. This binding of key to value in the leaf hash prevents second-preimage attacks where an adversary substitutes a different key at the same index.

d) Proof structure.: A Merkle proof consists of $d = 32$ sibling hashes and d direction bits, totaling $32 \times 32 + 4 = 1{,}028$ bytes. Verification recomputes the root by hashing the leaf with each sibling in sequence, following the direction bits from leaf to root.

C. Hash Function Selection

We select Poseidon [2] based on the following criteria:

1) **Constraint efficiency**: 240 R1CS constraints per 2-to-1 hash on BN254, the lowest among production-ready ZK hash functions [5].

2) **Field compatibility**: Operates natively over the BN128 scalar field, matching the field used by our Circom/SnarkJS Groth16 proof pipeline.
3) **Library availability**: circomlibjs (v0.1.7) provides a JavaScript implementation used by both the SMT and the existing `batch_verifier.circom` circuit.
4) **Production validation**: Used by Polygon zkEVM, Scroll, Iden3, and Semaphore.

For a depth-32 path verification in a Groth16 circuit, Poseidon requires $32 \times 240 = 7{,}680$ R1CS constraints, compared to $10{,}880$ for MiMC and an infeasible $4{,}800{,}000$ for Keccak-256.

IV. Experimental Methodology

A. Implementation

The SMT is implemented in TypeScript targeting Node.js v22.13.1. The implementation uses circomlibjs (v0.1.7) for Poseidon hashing, which internally operates on JavaScript BigInt values in the BN128 scalar field. The complete implementation comprises three components:

- `smt-implementation.ts`: Core SMT class (330 lines) with insert, update, delete, proof generation, and proof verification operations.
- `hash-comparison.ts`: Benchmark harness comparing Poseidon against a reference MiMC-Feistel implementation (x^7 S-box, 91 rounds).
- `smt-benchmark.ts`: SMT operation benchmark suite across four tree sizes.

B. Benchmark Configuration

a) Hash comparison.: 1,000 hash computations per function, preceded by 100 warmup iterations to allow JIT compilation. Measured: per-hash latency (μs), throughput (hashes/s). A 32-hash chain benchmark simulates Merkle path verification.

b) SMT operations.: Four tree sizes: $n \in \{100, 1{,}000, 10{,}000, 100{,}000\}$. For each size, 50 measurement samples are collected after 10 warmup iterations. Three operations are measured independently:

- **Insert latency**: Time to insert a single key-value pair, sampled uniformly across all insertions after warmup.
- **Proof generation**: Time to generate a Merkle proof for a random existing key (tree traversal only, no hashing).
- **Proof verification**: Time to verify a Merkle proof against the current root (32 Poseidon hashes).

Memory usage is measured via Node.js `process.memoryUsage().heapUsed`. Node counts reflect the number of non-default entries in the storage map.

c) Statistical reporting.: For each metric, we report mean, standard deviation, median (P50), 95th percentile (P95), and 99th percentile (P99). Key generation uses a deterministic linear congruential generator to ensure reproducibility.

C. Performance Targets

The performance targets derive from the enterprise validium use case, where the SMT is one component in a pipeline that must complete within enterprise SLA constraints:

TABLE II
Performance targets for SMT operations at $n = 100{,}000$ entries.

Metric	Target	Rationale
Insert latency	< 10 ms	Transaction ingestion rate
Proof generation	< 5 ms	Batch proving pipeline
Proof verification	< 2 ms	On-chain verification budget
Memory usage	< 2 GB	Enterprise server constraints
BN128 compatibility	Full	Circom circuit integration

D. Benchmark Validation

To ensure our measurements are credible, we reconcile results against published benchmarks from peer-reviewed sources and production systems. Any divergence exceeding one order of magnitude is investigated and documented. We note that direct JavaScript-to-Rust comparisons are expected to show large gaps ($100\times$–$1000\times$) due to BigInt overhead and absence of native field arithmetic in JavaScript; the relevant comparison is against the stated performance targets, not against native implementations in systems languages.

V. Results

All experiments were conducted on a Windows 11 x64 workstation running Node.js v22.13.1 with circomlibjs v0.1.7. Results are reported from the BN128 scalar field configuration.

A. Hash Function Performance

Table III presents the native JavaScript execution performance of Poseidon and MiMC over the BN128 scalar field, measured over 1,000 repetitions after 100 warmup iterations.

TABLE III
Hash function performance in native JavaScript (BN128 field).

Configuration	Latency (μs/hash)	Throughput (hash/s)
Poseidon 2-to-1	56.09	17,830
Poseidon 1-input	40.44	24,728
MiMC-Feistel (x^7, 91 rounds)	278.55	3,590
Poseidon chain-32	53.54/hash	18,677

Poseidon achieves a **4.97× speedup** over MiMC in native JavaScript execution. This is consistent with the theoretical ratio of constraint counts ($340/240 = 1.42\times$) amplified by MiMC's higher round count (91 rounds versus Poseidon's $\sim$65 effective rounds) and the quadratic cost of BigInt modular exponentiation for x^7 versus x^5.

A full depth-32 Merkle path verification requires 32 sequential Poseidon hashes, measured at 1.71 ms per chain (53.54 μs per individual hash within the chain). The slight reduction in per-hash latency within chains compared to isolated hashes (53.54 vs. 56.09 μs) is attributable to JIT optimization of the hot loop.

B. SMT Insert Latency

Table IV presents insert latency across four tree sizes, demonstrating the O(d) complexity where $d = 32$ is the tree depth.

TABLE IV
SMT INSERT LATENCY BY TREE SIZE (50 SAMPLES PER CONFIGURATION).

Entries	Mean (ms)	P50 (ms)	P95 (ms)	Stddev (ms)
100	1.877	1.864	2.087	0.134
1,000	1.788	1.780	1.997	0.095
10,000	1.792	1.777	1.946	0.085
100,000	1.825	1.811	2.014	0.098

Insert latency is effectively constant across tree sizes at approximately 1.8 ms, confirming the O(d) complexity prediction. The latency is dominated by 32 Poseidon hash computations ($\sim$1.7 ms) plus hash map lookups for sibling retrieval. The slight variation across sizes (1.788–1.877 ms) falls within one standard deviation and is attributable to JIT warmup effects and V8 garbage collection.

C. Proof Generation Time

Table V shows proof generation times, which involve tree traversal without hash computation.

TABLE V
SMT PROOF GENERATION TIME BY TREE SIZE (50 SAMPLES PER CONFIGURATION).

Entries	Mean (ms)	P95 (ms)
100	0.012	0.029
1,000	0.014	0.016
10,000	0.014	0.016
100,000	0.018	0.021

Proof generation is extremely fast ($<$ 0.02 ms at 100K entries) because it only traverses the tree to collect the 32 sibling hashes—no hash computation is required. This is 278$\times$ faster than the 5 ms target. The marginal increase from 0.012 ms (100 entries) to 0.018 ms (100K entries) reflects the larger hash map size increasing lookup times.

D. Proof Verification Time

Table VI presents verification times, which require 32 sequential Poseidon hashes.

TABLE VI
SMT PROOF VERIFICATION TIME BY TREE SIZE (50 SAMPLES PER CONFIGURATION).

Entries	Mean (ms)	P95 (ms)
100	1.685	1.788
1,000	1.677	1.758
10,000	1.719	1.905
100,000	1.744	1.869

Verification time is constant at approximately 1.7 ms across all tree sizes, as expected (always 32 Poseidon hashes regardless of population). This is consistent with the chain-32 benchmark (1.71 ms) from Section V.A. The P95 at 100K entries (1.869 ms) is within the 2 ms target with a 7% margin.

E. Memory Usage

Table VII summarizes memory consumption across tree sizes.

TABLE VII
MEMORY USAGE BY TREE SIZE.

Entries	Nodes Stored	Heap (MB)	Bytes/Entry
100	2,721	68.4	$\sim$700,000
1,000	23,867	112.8	$\sim$115,000
10,000	202,398	57.9	$\sim$5,900
100,000	1,712,112	233.9	$\sim$2,400

Node count scales at approximately 17$\times$ the entry count, consistent with each insert creating or updating up to 33 nodes (1 leaf + 32 internal path nodes). At 100K entries, memory usage of 233.9 MB is well within the 2 GB threshold. The amortized bytes-per-entry decreases with tree size due to fixed V8 runtime overhead being distributed across more entries. The anomalous 57.9 MB reading at 10K entries is attributed to V8 garbage collection between benchmark runs (each tree size uses a fresh SMT instance).

F. Benchmark Reconciliation

Table VIII compares our measurements against published reference values.

TABLE VIII
BENCHMARK RECONCILIATION WITH PUBLISHED DATA.

Metric	Ours	Published	Assessment
Poseidon vs MiMC	4.97$\times$	$\sim$2$\times$ [19]	Consistent (circuit vs. native)
SMT insert	1.825 ms	1.92 μs [15]	Expected: JS vs. Rust
Proof generation	0.018 ms	20–83 μs [15]	Comparable
Poseidon constraints	240	240 [5]	Exact match

The 950$\times$ gap between our JavaScript insert latency and Rust implementations is expected and not concerning: Rust uses native 256-bit arithmetic while JavaScript relies on BigInt with heap-allocated arbitrary-precision integers. The relevant comparison is against the stated performance targets, all of which pass.

VI. DISCUSSION

A. Hypothesis Assessment

The experimental hypothesis stated that a depth-32 SMT with Poseidon hash could support 100,000+ entries with insert latency $<$ 10 ms, proof generation $<$ 5 ms, and proof verification $<$ 2 ms in TypeScript, while maintaining BN128 field compatibility. All five criteria are met:

TABLE IX
HYPOTHESIS EVALUATION AT $n = 100{,}000$ ENTRIES.

Target	Required	Measured (P95)	Margin
Insert latency	$<$ 10 ms	2.014 ms	5.0$\times$
Proof generation	$<$ 5 ms	0.021 ms	238$\times$
Proof verification	$<$ 2 ms	1.869 ms	1.07$\times$
Memory	$<$ 2 GB	233.9 MB	8.5$\times$
BN128 compatibility	Full	Full	—

The hypothesis is **confirmed**. However, the margins are not uniform, and the tightest constraint (proof verification at 1.07$\times$ P95 margin) warrants discussion.

B. Trade-off Analysis

a) Proof verification is the bottleneck.: Verification requires 32 sequential Poseidon hashes with no opportunity for parallelism or caching, making it the irreducible cost floor of the system. The P95 of 1.869 ms leaves only 131 μs of headroom against the 2 ms target. On slower hardware or under concurrent load, this margin could vanish. Two mitigation paths exist: (1) WebAssembly Poseidon implementations (e.g., circom-compat-wasm) offer approximately 10× speedup over JavaScript BigInt by using native 64-bit arithmetic, and (2) a native Node.js addon in Rust or C++ could achieve 100× improvement.

b) Memory scales linearly.: At 100K entries, the implementation uses 233.9 MB. Extrapolating the 17× node-to-entry ratio, 1M entries would require approximately 2.3 GB, exceeding the 2 GB threshold. For deployments beyond 100K entries, the in-memory `Map` should be replaced with a persistent key-value store (LevelDB or RocksDB) with an LRU cache for hot path nodes. This is an implementation concern, not an algorithmic one—the O(d) complexity is unaffected.

c) Insert latency is dominated by hashing.: The 1.825 ms mean insert latency comprises approximately 1.7 ms of Poseidon computation (32 hashes) and approximately 0.1 ms of hash map operations. This means insert performance cannot be significantly improved without changing the hash implementation. For the batch processing pipeline (where the SMT receives transactions in batches of 4–256), the batch update optimization of Chen et al. [14] can reduce amortized cost by sharing common path computation across multiple inserts.

d) Poseidon vs. MiMC: a settled question.: Our 4.97× native speedup, combined with the 29% constraint reduction (240 vs. 340 R1CS), the production consensus (Polygon, Scroll, Iden3, Semaphore all use Poseidon), and identical BN128 field compatibility, leaves no technical argument for MiMC in this context. The only caveat is that MiMC may perform better under Plonkish arithmetization [5], which is irrelevant to our Groth16/R1CS pipeline.

C. Limitations

a) Single-run, single-machine.: All benchmarks were conducted on a single Windows 11 workstation. Results may differ on Linux servers with different V8 JIT behavior, NUMA architectures, or containerized environments. The 50-sample methodology provides statistical confidence for this configuration but not generalizability.

b) Synthetic workload.: Keys are generated by a deterministic linear congruential generator, producing uniform distribution. Enterprise workloads may exhibit locality (e.g., sequential record IDs for a single enterprise), which could affect hash map performance. The tree structure itself is unaffected by key distribution due to the Poseidon key derivation, which distributes any input uniformly.

c) No concurrent access.: The implementation is single-threaded with no locking or transaction isolation. A production deployment processing concurrent transactions will require a concurrency control layer above the SMT.

d) JavaScript BigInt overhead.: Our measurements reflect JavaScript's arbitrary-precision BigInt, which is orders of magnitude slower than native field arithmetic in Rust or C++. The absolute numbers (1.8 ms insert, 1.7 ms verify) should be interpreted as upper bounds; a native implementation would likely achieve 10×–100× better performance.

D. Reconciliation with Literature

The 4.97× Poseidon-over-MiMC speedup in native execution is higher than the ∼2× proving time improvement reported by Semaphore [19]. This is expected: Semaphore measures end-to-end circuit proving time (which includes witness generation, FFT, and multi-scalar multiplication in addition to hash computation), while we isolate native hash throughput. The ratio of constraint counts ($340/240 = 1.42\times$) represents a lower bound on the speedup; native execution amplifies this because MiMC's x^7 S-box requires more BigInt multiplications per round than Poseidon's x^5.

The 950× gap between our JavaScript insert latency and Rust (Monotree: 1.92 μs) is consistent with known BigInt-to-native arithmetic overhead. The Reilabs LargeSMT [15] multi-threaded numbers (63–193 μs) narrow this to 9–29× when comparing single-threaded JavaScript against multi-threaded Rust, which remains consistent with the expected overhead range.

VII. Conclusion

This paper presented the design, implementation, and empirical evaluation of a depth-32 Sparse Merkle Tree with Poseidon hash for enterprise ZK validium state management on the Basis Network. Our results demonstrate that this configuration meets all performance requirements for the enterprise use case:

- Insert latency of 1.825 ms mean (5.5× below the 10 ms target), constant across tree sizes from 100 to 100,000 entries, confirming O(d) complexity.
- Proof generation in 0.018 ms mean (278× below the 5 ms target), achieved through traversal-only proof collection.
- Proof verification in 1.744 ms mean (within the 2 ms target), dominated by 32 sequential Poseidon hashes.
- Memory usage of 233.9 MB at 100,000 entries, well within enterprise server constraints.
- Full BN128 field compatibility via circomlibjs, enabling direct integration with Circom/SnarkJS Groth16 circuits.

The hash function comparison confirms Poseidon as the definitive choice for ZK-friendly state trees, with a 4.97× native speedup over MiMC and 29% fewer R1CS constraints per hash, consistent with production system choices across the industry.

A. Future Work

Three directions emerge from this evaluation:

a) Performance optimization.: Replacing JavaScript BigInt with WebAssembly or native Poseidon implementations can reduce verification latency by 10–100×, eliminating the tightest performance margin. This is a straightforward engineering task with no algorithmic changes required.

b) Persistent storage.: For deployments exceeding 100,000 entries, migrating the in-memory hash map to a database-backed store (LevelDB or RocksDB) with path-aware caching will enable scaling to millions of entries without memory constraints.

c) Batch update optimization.: Integrating the one-phase batch update technique of Chen et al. [14] will reduce amortized insert cost for the batch aggregation pipeline, where multiple transactions share common tree paths.

These results establish the SMT as a validated foundation for the enterprise validium pipeline. The implementation serves as the reference design for the production Architect agent and the formal specification target for TLA+ model checking in subsequent pipeline stages.

REFERENCES

[1] J. Groth, "On the size of pairing-based non-interactive arguments," in *Advances in Cryptology – EUROCRYPT 2016*, ser. LNCS, vol. 9666. Springer, 2016, pp. 305–326.

[2] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger, "Poseidon: A new hash function for zero-knowledge proof systems," in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 2021, pp. 519–535, iACR ePrint 2019/458.

[3] M. Albrecht, L. Grassi, C. Rechberger, A. Roy, and T. Tiessen, "MiMC: Efficient encryption and cryptographic hashing with minimal multiplicative complexity," in *Advances in Cryptology – ASIACRYPT 2016*, ser. LNCS, vol. 10031. Springer, 2016, pp. 191–219.

[4] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger, "Poseidon2: A faster version of the Poseidon hash function," *IACR ePrint 2023/323*, 2023.

[5] J. Guohanze, Y. Chen, and L. Zhang, "Benchmarking ZK-friendly hash functions and SNARK proving systems for EVM-compatible blockchains," *arXiv preprint arXiv:2409.01976*, 2024.

[6] C. Nielsen, "SNARK-hash-benchmark," GitHub, 2023.

[7] L. Grassi *et al.*, "Poseidon and Neptune: Gröbner basis cryptanalysis exploiting subspace trails," *IACR ePrint 2025/954*, 2025.

[8] Ethereum Foundation, "Poseidon cryptanalysis initiative 2024–2026," poseidon-initiative.info, 2024.

[9] A. Aly, T. Ashur, E. Ben-Sasson, S. Dhooghe, and A. Szepieniec, "Design of symmetric-key primitives for advanced cryptographic protocols," in *IACR Transactions on Symmetric Cryptology*, vol. 2020, no. 3, 2020, pp. 1–45.

[10] R. C. Merkle, "A digital signature based on a conventional encryption function," *Advances in Cryptology – CRYPTO '87*, pp. 369–378, 1987.

[11] R. Dahlberg, T. Pulls, and R. Peeters, "Efficient sparse Merkle trees: Caching strategies and secure (non-)membership proofs," *IACR ePrint 2016/683*, 2016.

[12] F. Haider, "Compact sparse Merkle trees," *IACR ePrint 2018/955*, 2018.

[13] J. Baylina and M. Belles, "Sparse Merkle trees," in *ZKProof Standards Workshop 2*. Iden3, 2019.

[14] W. Chen, J. Park, and Y. Lee, "One-phase batch update on sparse Merkle trees for rollups," *arXiv preprint arXiv:2310.13328*, 2023.

[15] Reilabs, "Scaling sparse Merkle trees to billions of keys with LargeSMT," Blog post, 2024.

[16] Polygon Labs, "Sparse Merkle tree – Polygon zkEVM documentation," https://docs.polygon.technology/, 2024.

[17] Scroll, "Scroll architecture: Poseidon sparse Merkle tree," https://docs.scroll.io/, 2024.

[18] Iden3, "Sparse Merkle tree implementation," GitHub: iden3/go-merkletree-sql, 2023.

[19] Semaphore Protocol, "Release v4.0.0," GitHub, 2024.